

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Операционные системы»
Тема: Межпроцессные взаимодействия,
(разделение памяти+синхронизация, сокеты: UDP, TCP, сравнение)

Студентка гр. 2304

Иванова М.А.

Преподаватель

Душутина Е.В.

Санкт-Петербург

2024

Цель работы.

Изучить межпроцессные взаимодействия в Unix-подобных ОС. Изучить разделение памяти и синхронизацию при межпроцессных взаимодействиях Linux, а также сокеты UDP и TCP, сравнить их. Научиться применять перечисленные выше операции и файлы на практике.

Задание.

8. Каналы: реализуйте pipe и
 1. fifo. Приведите в отчете фрагмент файла, содержащего ограничения для IPC (для каналов и сообщений).
 2. Осуществите передачу информации посредством обмена сообщениями по принципу «почтового ящика», т.е. не синхронизируя отправителя и получателя (без ожидания доставки).
 3. Организуйте обмен сообщениями так, чтобы некоторому событию соответствовал отдельный тип сообщения. (Для реализации можно, например, использовать функции eventfd, poll)
 4. Выполните передачу информации локально посредством сокетов по TCP/IP,
 5. а затем в сетевом режиме (посредством сокетов по TCP/IP)
 6. Организуйте взаимодействие с 10, 100 и 1000 клиентами в клиент-серверном приложении (посредством сокетов). Оцените ограничения
 7. Выполните аналогичное взаимодействие на основе UDP,
 8. экспериментально продемонстрируйте разницу между TCP и UDP реализациями
 9. Обеспечьте разделение памяти между независимыми процессами и необходимую синхронизацию для эффективного взаимодействия

Выполнение работы.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~$ neofetch
...-:~::~-...
.-MMMMMMMMMMMMMMMM-
.-MMM`...-:~::~-`MMM-
.:MMMM.:MMMMMMMMMMMMMMMM.:MMM:.
-MMM-M--MMMMMMMMMMMMMMMMMMMM.MMM-
`.:MMM:MM`.:MMM:.....-MMM:MMM:`
:MMM:MMM`.:MM:`\`.\`.\`.:MMM:MMM:
.MMM.MMMM`.:MM.-MM-.MM-`MMMM.MMM.
:MMM:MMM`.:MM.-MM-.MM:.`MMMM-MMM:
:MMM:MMM`.:MM.-MM-.MM:.`MMMM-MMM:
:MMM:MMM`.:MM.-MM-.MM:.`MMMM-MMM:
:MMM.MMMM`.:MM:--MM:--MM:.`MMMM.MMM.
:MMM:MMM-`-MMMMMMMMMMMMMM-`-MMM-MMM:
:MMM:MMM:`.:MMM:MMM:
.MMM.MMMM:-----MMMM.MMM.
'-MMM.-MMMMMMMMMMMMMMMM-.'
'..-MMM`-:~::~-`MMM-'
'-MMMMMMMMMMMMMM-'
`-:~::~-`

OS: Linux Mint 21.3 x86_64
Host: 82JD Lenovo Legion Y9000P2021H
Kernel: 5.15.0-100-generic
Uptime: 3 hours, 36 mins
Packages: 2507 (dpkg), 8 (flatpak)
Shell: bash 5.1.16
Resolution: 2928x1830
DE: Cinnamon 6.0.4
WM: Mutter (Muffin)
WM Theme: Mint-L-Dark-Purple (Mint-Y
Theme: Mint-L-Dark-Purple [GTK2/3]
Icons: Mint-L-Purple [GTK2/3]
Terminal: gnome-terminal
CPU: 11th Gen Intel i7-11800H (16) @
GPU: Intel TigerLake-H GT1 [UHD Grap
GPU: NVIDIA GeForce RTX 3060 Mobile
Memory: 6176MiB / 31875MiB
```

Рис. 1 — Информация о системе

8. Задание: реализуйте канал pipe.

Неименованные каналы создаются посредством системного вызова `int pipe(int fd[2])`, который возвращает 0 в случае успешного завершения. (– 1) – в случае ошибки и 2 файловых дескриптора: `filedes[1]` для записи в канал и `filedes[0]` для чтения из канала.

Была написана программа, создающая неименованный канал. Родительский процесс записывает данные из файла для чтения в канал, а дочерний процесс читает данные из канала и записывает их в файл для записи.

Исходный код: Файл 8.c

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <strings.h>

#include <unistd.h>

#include <wait.h>


int main() {

    // файл для записи

    char fileToRead[] = "readFile.txt";

    // файл для чтения

    char fileToWrite[] = "writeFile.txt";

    // дескрипторы для неименованного канала

    int descriptors[2];

    // создаем неименованный канал

    if (pipe(descriptors) < 0) {

        printf("Не удалось создать неименованный канал\n");

        exit(1);

    }

    printf("Создан неименованный канал\n");


    // дочерний процесс

    if (fork() == 0) {

        // закрываем неименованный канал на чтение - будем только
        писать

        close(descriptors[0]);

        // открываем файл на чтение

        FILE *f = fopen(fileToRead, "r");

        if (f == NULL) {

            printf("Не удалось открыть файл для чтения\n");

            exit(1);

        }

    }

}
```

```

    }

    printf("Дочерний процесс читает данные из файла %s и записывает
их в канал\n", fileToRead);

    // сообщение
pitatir@pitatir-Lenovo-Legion-
Y9000P2021H:~/LR5/9$ gcc client.c -o c

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/9$ ./c

Открываем канал 1 для чтения

Открываем канал 2 для записи

Читаем имя файла из канала 1

Открываем файл buffer.txt

Читаем данные из файла и пишем в канал 2

Закрываем каналы

Работа клиента завершена

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/9$

```

Как видно из логов, программы успешно осуществляют передачу данных через именованные каналы.

Изучим файлы с ограничениями:

Максимальный размер буфера канала:

/proc/sys/fs/pipe-max-size - 1048576

Максимальный размер сообщения:

/proc/sys/kernel/msgmax - 8192

Максимальное количество сообщений в очереди:

/proc/sys/kernel/msgmnb - 16384

Максимальный размер очереди сообщений:

/proc/sys/kernel/msgmni - 32000

10. **Задание:**

Осуществите передачу информации посредством обмена сообщениями по принципу «почтового ящика», т.е. не синхронизируя отправителя и получателя (без ожидания доставки).

Решение:

Были написаны 2 программы: сервер, который читает сообщения из очереди с задержкой в 3 секунды, и клиент, который отправляет считанные из консоли сообщения в очередь. В обеих программах есть пользовательский обработчик сигнала SIGINT, корректно завершающий работу программы (в программе-сервере обработчик удаляет очередь сообщений).

Обе программы обращаются к переданному в качестве аргумента файлу или к файлу "key.txt" в противном случае для получения уникального ключа доступа к очереди сообщений.

Исходный код: Файл server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>

#define DEF_KEY_FILE "key.txt"
```

```

// Структура сообщения
typedef struct {
    // тип сообщения
    long type;
    // текст сообщения
    char buf[100];
} Message;

int queue;

// Обработчик сигнала SIGINT
void intHandler(int sig) {
    // вызываем обработчик по умолчанию
    signal(sig, SIG_DFL);
    // удаление очереди сообщений
    if (msgctl(queue, IPC_RMID, 0) < 0) {
        printf("Ошибка удаления очереди\n");
        exit(1);
    }
    printf("\nРабота сервера завершена\n");
    exit(0);
}

int main(int argc, char **argv) {
    // имя файла с ключом
    char keyFile[100];
    // очищаем буфер
    bzero(keyFile, 100);
    // если не передан файл с ключом
    if (argc < 2) {
        printf("Используем файл с ключом по умолчанию %s\n",
DEF_KEY_FILE);
        strcpy(keyFile, DEF_KEY_FILE);
    } else
        // в противном случае используем переданный файл
        strcpy(keyFile, argv[1]);
    // уникальный ключ, необходимый для задания очереди
    key_t key;
    // получение ключа
    // преобразование имени файла и идентификатора
    key = ftok(keyFile, 'Q');

```

```

// в случае ошибки
if (key == -1) {
    printf("Не получен ключ для файла %s и ID 'Q'\n", keyFile);
    exit(1);
}
// создание очереди сообщений
queue = msgget(key, IPC_CREAT | 0666);
// в случае ошибки
if (queue < 0) {
    printf("Ошибка создания очереди\n");
    exit(4);
}

// связывание сигнала с его обработчиком
signal(SIGINT, intHandler);

Message mes;
int res;
// бесконечный цикл чтения сообщений
for (;;) {
    // очищение буфера
    bzero(mes.buf, 100);
    // чтение сообщения
    res = (int) msgrcv(queue, &mes, sizeof(Message), 1L, 0);
    // в случае ошибки
    if (res < 0) {
        printf("Ошибка получения сообщения\n");
        kill(getpid(), SIGINT);
    }
    // вывод сообщения
    printf("Сообщение клиента: %s\n", mes.buf);
    sleep(3);
}
return 0;
}

```

Файл client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>

```



```

#include <sys/msg.h>
#include <string.h>
#include <signal.h>

#define DEF_KEY_FILE "key.txt"

// Структура сообщения
typedef struct {
    // тип сообщения
    long type;
    // текст сообщения
    char buf[100];
} Message;

int queue;

// Обработчик сигнала SIGINT
void intHandler(int sig) {
    // вызываем обработчик по умолчанию
    signal(sig, SIG_DFL);
    printf("\nРабота клиента завершена\n");
    exit(0);
}

int main(int argc, char **argv) {
    // имя файла с ключом
    char keyFile[100];
    // очищаем буфер
    bzero(keyFile, 100);
    // если не передан файл с ключом
    if (argc < 2) {
        printf("Используем файл с ключом по умолчанию %s\n",
DEF_KEY_FILE);
        strcpy(keyFile, DEF_KEY_FILE);
    } else
        // в противном случае используем переданный файл
        strcpy(keyFile, argv[1]);
    // уникальный ключ, необходимый для задания очереди
    key_t key;
    // получение ключа
    // преобразование имени файла и идентификатора

```

```

key = ftok(keyFile, 'Q');
// в случае ошибки
if (key == -1) {
    printf("Не получен ключ для файла %s и ID 'Q'\n", keyFile);
    exit(1);
}
// получение доступа к созданной сервером очереди сообщений
queue = msgget(key, 0);
// в случае ошибки
if (queue < 0) {
    printf("Ошибка доступа к очереди\n");
    exit(4);
}

// связывание сигнала с его обработчиком
signal(SIGINT, intHandler);

Message mes;
int res;
// бесконечный цикл отправки сообщений
for (;;) {
    // задаем сообщению тип 1
    mes.type = 1L;
    // очищение буфера
    bzero(mes.buf, 100);
    // чтение сообщения из консоли
    fgets(mes.buf, 100, stdin);
    // дописываем символ конца строки
    mes.buf[strlen(mes.buf) - 1] = '\0';
    // отправляем это сообщение
    res = msgsnd(queue, (void *) &mes, sizeof(Message), 0);
    // в случае ошибки
    if (res != 0) {
        printf("Ошибка отправки сообщения\n");
        exit(1);
    }
}
return 0;
}

```

В одном терминале была запущена программа-сервер, а еще в двух терминалах - программы-клиенты.

Лог выглядит следующим образом:

Терминал с сервером:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$ gcc server.c -
o s
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$ gcc client.c -
o s
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$ ./s
Используем файл с ключом по умолчанию key.txt
Сообщение клиента: 1
Сообщение клиента: 2
Сообщение клиента: 3
Сообщение клиента: 4
Сообщение клиента: 5
Сообщение клиента: 6
Сообщение клиента: 7
Сообщение клиента: 8
^C
Работа сервера завершена
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$
```

Терминал с клиентом 1:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$ ./c
Используем файл с ключом по умолчанию key.txt
1
2
5
8
9
10
Ошибка отправки сообщения
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$
```

Терминал с клиентом 2:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10$ ./c
Используем файл с ключом по умолчанию key.txt
3
4
```

6

7

^C

Работа клиента завершена

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/10\$

Важно отметить порядок ввода сообщений:

- Сообщения вводились в порядке, соответствующем цифрам: первые два сообщения отправил клиент 1, потом еще два - клиент 2, затем сообщение “5” отправил клиент 1 и т.д.
- Сервер считывает сообщения из очереди с задержкой, и в момент прерывания работы программы-клиента 2, сервер не успел считать все отправленные клиентом 2 сообщения (т.е. работа клиента 2 была прекращена до вывода сервером сообщения “7”). Тем не менее, все отправленные клиентом 2 сообщения были выведены на экран - это говорит о том, что с завершением процесса-источника данные не исчезают из очереди, и что очередь сообщений находится в адресном пространстве ядра.
- Работа программы-сервера была прекращена сразу после ввода клиентом 1 сообщения “9”, поэтому сервер не успел считать его из очереди, хотя она была не пуста. Этот пункт показывает отсутствие синхронизации отправителя и получателя, как и требуется по условию задания.
- После завершения работы программы-сервера клиент 1 попытался отправить сообщение “10”, что привело к ошибке: клиент 1 попытался

получить доступ к очереди сообщений после ее удаления сервером в момент завершения работы.

Лог отражает успешную передачу сообщений между клиентами и сервером по принципу «почтового ящика» без ожидания доставки сообщения.

11. **Задание:** Организуйте обмен сообщениями так, чтобы некоторому событию соответствовал отдельный тип сообщения. (Для реализации можно, например, использовать функции `eventfd`, `poll`)

Решение:

Были написаны 3 программы:

1. Программа-сервер, которая считывает сообщения из очереди и выводит их на экран. В программе также предусмотрен обработчик сигнала `SIGINT`, который удаляет очередь сообщений при завершении работы программы.
2. Программа-клиент 1, которая считывает сообщения из консоли и записывает их в очередь. Для возможности отслеживания событий с помощью `epoll()` в момент начала работы программы открывается именной канал, в который записывается сообщение. Канал закрывается вместе с завершением работы программы.
3. Программа-клиент 2, созданная для отслеживания событий в клиенте 1 и отправки сообщений об этом. Клиент 2 с помощью `wait_epoll()`

следит за событиями EPOLLIN (доступны данные для чтения) и EPOLLHUP (связь потеряна) с в канале 1. При срабатывании события EPOLLIN отправляется сообщение типа 2, уведомляющее об подключении клиента 1, а при событии EPOLLHUP - сообщение типа 3 о завершении работы клиента 1.

Стоит отметить, что программа-клиент 2 запускается до клиента 1.

Исходный код: Файл server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>

#define DEF_KEY_FILE "key.txt"

// Структура сообщения
typedef struct {
    // тип сообщения
    long type;
    // текст сообщения
    char buf[100];
} Message;

int queue;

// Обработчик сигнала SIGINT
void intHandler(int sig) {
    // вызываем обработчик по умолчанию
    signal(sig, SIG_DFL);
    // удаление очереди сообщений
    if (msgctl(queue, IPC_RMID, 0) < 0) {
        printf("Ошибка удаления очереди\n");
        exit(1);
    }
}
```

```

    printf("\nРабота сервера завершена\n");
    exit(0);
}

int main(int argc, char **argv) {
    // имя файла с ключом
    char keyFile[100];
    // очищаем буфер
    bzero(keyFile, 100);
    // если не передан файл с ключом
    if (argc < 2) {
        printf("Используем файл с ключом по умолчанию %s\n",
DEF_KEY_FILE);
        strcpy(keyFile, DEF_KEY_FILE);
    } else
        // в противном случае используем переданный файл
        strcpy(keyFile, argv[1]);
    // уникальный ключ, необходимый для задания очереди
    key_t key;
    // получение ключа
    // преобразование имени файла и идентификатора
    key = ftok(keyFile, 'Q');
    // в случае ошибки
    if (key == -1) {
        printf("Не получен ключ для файла %s и ID 'Q'\n", keyFile);
        exit(1);
    }
    // создание очереди сообщений
    queue = msgget(key, IPC_CREAT | 0666);
    // в случае ошибки
    if (queue < 0) {
        printf("Ошибка создания очереди\n");
        exit(4);
    }

    // связывание сигнала с его обработчиком
    signal(SIGINT, intHandler);

    Message mes;
    int res;

```

```

// бесконечный цикл чтения сообщений
for (;;) {
    // очищение буфера
    bzero(mes.buf, 100);
    // чтение сообщения
    res = (int) msgrcv(queue, &mes, sizeof(Message), 0, 0);
    // в случае ошибки
    if (res < 0) {
        printf("Ошибка получения сообщения\n");
        kill(getpid(), SIGINT);
    }
    // вывод сообщения
    printf("Сообщение клиента типа %ld: %s \n", mes.type,
mes.buf);
}
return 0;
}

```

Файл client1.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
#include <signal.h>
#include <fcntl.h>
#include <unistd.h>

#define DEF_KEY_FILE "key.txt"

// Структура сообщения
typedef struct {
    // тип сообщения
    long type;
    // текст сообщения
    char buf[100];
} Message;

// идентификатор очереди сообщений
int queue;
// дескриптор канала 1

```



```

int chan1;

// Обработчик сигнала SIGINT
void intHandler(int sig) {
    // вызываем обработчик по умолчанию
    signal(sig, SIG_DFL);
    printf("\nРабота клиента 1 завершена\n");
    unlink("channel1");
    close(chan1);
    exit(0);
}

int main(int argc, char **argv) {
    // имя файла с ключом
    char keyFile[100];
    // сообщение
    Message mes;
    // возвращаемое функцией значение
    int res;

    // ПОЛУЧЕНИЕ ДОСТУПА К ОЧЕРЕДИ С ПОМОЩЬЮ КЛЮЧА
    // очищаем буфер
    bzero(keyFile, 100);
    // если не передан файл с ключом
    if (argc < 2) {
        printf("Используем файл с ключом по умолчанию %s\n",
DEF_KEY_FILE);
        strcpy(keyFile, DEF_KEY_FILE);
    } else
        // в противном случае используем переданный файл
        strcpy(keyFile, argv[1]);
    // уникальный ключ, необходимый для задания очереди
    key_t key;
    // получение ключа
    // преобразование имени файла и идентификатора
    key = ftok(keyFile, 'Q');
    // в случае ошибки
    if (key == -1) {
        printf("Не получен ключ для файла %s и ID 'Q'\n", keyFile);
        exit(1);
    }
}

```

```

// получение доступа к созданной сервером очереди сообщений
queue = msgget(key, 0);
// в случае ошибки
if (queue < 0) {
    printf("Ошибка доступа к очереди\n");
    exit(4);
}

// ПРИВЯЗЫВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ОБРАБОТЧИКА СИГНАЛА SIGINT
signal(SIGINT, intHandler);

// ИСПОЛЬЗОВАНИЕ КАНАЛА 1 КАК ПРИЗНАК НАЧАЛА РАБОТЫ КАНАЛА 1
// получаем дескриптор канала
chan1 = open("channel1", O_RDWR);
if (chan1 == -1) {
    printf("Ошибка открытия канала channel1\n");
    // удаляем первый канал
    unlink("channel1");
    exit(1);
}
printf("Записываем в канал 1 \"client1\"\n");
// пишем имя файла в канал 1
write(chan1, "client1", strlen("client1"));

// ОТПРАВКА СЕРВЕРУ СООБЩЕНИЙ, ВВЕДЕННЫХ ПОЛЬЗОВАТЕЛЕМ
// бесконечный цикл отправки сообщений
for (;;) {
    // задаем сообщению тип 1
    mes.type = 1L;
    // очищение буфера
    bzero(mes.buf, 100);
    // чтение сообщения из консоли
    fgets(mes.buf, 100, stdin);
    // дописываем символ конца строки
    mes.buf[strlen(mes.buf) - 1] = '\0';
    // отправляем это сообщение
    res = msgsnd(queue, (void *) &mes, sizeof(Message), 0);
    // в случае ошибки
    if (res != 0) {
        printf("Ошибка отправки сообщения\n");
    }
}

```

```

        unlink("channel1");
        close(chan1);
        exit(1);
    }
}
return 0;
}

```

Файл client2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/epoll.h>
#include <fcntl.h>
#include <unistd.h>

#define DEF_KEY_FILE "key.txt"

// Структура сообщения
typedef struct {
    // тип сообщения
    long type;
    // текст сообщения
    char buf[100];
} Message;

// идентификатор очереди сообщений
int queue;
// дескриптор канала 1
int chan1;

// Обработчик сигнала SIGINT
void intHandler(int sig) {
    // вызываем обработчик по умолчанию
    signal(sig, SIG_DFL);
    printf("\nРабота клиента 2 завершена\n");
    unlink("channel1");
}

```

```

    close(chan1);
    exit(0);
}

int main(int argc, char **argv) {
    // имя файла с ключом
    char keyFile[100];
    // сообщение
    Message mes;
    // возвращаемое функцией значение
    int res;

    // ПОЛУЧЕНИЕ ДОСТУПА К ОЧЕРЕДИ С ПОМОЩЬЮ КЛЮЧА
    // очищаем буфер
    bzero(keyFile, 100);
    // если не передан файл с ключом
    if (argc < 2) {
        printf("Используем файл с ключом по умолчанию %s\n",
DEF_KEY_FILE);
        strcpy(keyFile, DEF_KEY_FILE);
    } else
        // в противном случае используем переданный файл
        strcpy(keyFile, argv[1]);
    // уникальный ключ, необходимый для задания очереди
    key_t key;
    // получение ключа
    // преобразование имени файла и идентификатора
    key = ftok(keyFile, 'Q');
    // в случае ошибки
    if (key == -1) {
        printf("Не получен ключ для файла %s и ID 'Q'\n", keyFile);
        exit(1);
    }
    // получение доступа к созданной сервером очереди сообщений
    queue = msgget(key, 0);
    // в случае ошибки
    if (queue < 0) {
        printf("Ошибка доступа к очереди\n");
        exit(4);
    }
}

```

```

// ПРИВЯЗЫВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ОБРАБОТЧИКА СИГНАЛА SIGINT
signal(SIGINT, intHandler);

// СОЗДАНИЕ КАНАЛА 1 ДЛЯ ИНДИКАЦИИ РАБОТЫ КЛИЕНТА 1
printf("Создание канала 1 для индикации работы клиента 1\n");
// создаем первый именованный канал
// устанавливаем для всех права на чтение и на запись
res = mknod("channel1", S_IFIFO | 0666, 0);
// в случае ошибки
if (res) {
    printf("Ошибка создания канала 1\n");
    // удаляем первый канал
    unlink("channel1");
    exit(1);
}

// МОНИТОРИНГ КЛИЕНТА 1 ЧЕРЕЗ КАНАЛ 1
// получаем дескриптор файла
chan1 = open("channel1", O_RDONLY);
// в случае ошибки
if (chan1 == -1) {
    printf("Ошибка открытия канала channel1\n");
    unlink("channel1");
    exit(1);
}

// создаем структуру события для отслеживания
struct epoll_event event;
// задаем дескриптор для отслеживания
event.data.fd = chan1;
// задаем событие для отслеживания
event.events = EPOLLIN | EPOLLHUP;

// создание файлового дескриптора epoll
int epoll_fd = epoll_create1(0);
// в случае ошибки
if (epoll_fd == -1) {
    printf("Ошибка создания файлового дескриптора epoll\n");
    unlink("channel1");
    close(chan1);
}

```

```

        exit(1);
    }

    // связываем структуру event с файлом
    int ret = epoll_ctl(epoll_fd, EPOLL_CTL_ADD, chan1, &event);
    // в случае ошибки
    if (ret == -1) {
        printf("Ошибка epoll_ctl\n");
        unlink("channel1");
        close(chan1);
        close(epoll_fd);
        exit(1);
    }

    // флаги, отвечающие за единичный вывод сообщения о
событии
    int epollin = 0;
    int epollhup = 0;
    // бесконечный цикл
    while (1) {
        // хранение отслеживаемых событий
        struct epoll_event events[2];
        // количество реально произошедших отслеживаемых событий
        int num_events = epoll_wait(epoll_fd, events, 2, -1);
        // в случае ошибки
        if (num_events == -1) {
            printf("Ошибка epoll_wait\n");
            unlink("channel1");
            close(chan1);
            close(epoll_fd);
            exit(1);
        }
        // проверяем все произошедшие события
        for (int i = 0; i < num_events; i++) {
            // если произошло событие EPOLLHUP
            char buf[100];

            // пока не закончится файл
            while (!feof(f)) {

                // читаем данные из файла

```

```

        int n = (int) fread(buf, sizeof(char), 100, f);

        // записываем в неименованный канал
        write(descriptors[1], buf, n);
    }

    // закрываем файл
    fclose(f);

    // закрываем неименованный канал
    close(descriptors[1]);

    return 0;
}

// родительский процесс

// закрываем неименованный канал на запись - будем только
читать

close(descriptors[1]);

// открываем файл
FILE *f = fopen(fileToWrite, "w");

if (f == NULL) {
    printf("Не удалось открыть файл для записи\n");
    exit(1);
}

wait(NULL);

// сообщение

char buf[100];

printf("Родительский процесс читает данные из канала и
записывает их в файл %s\n", fileToWrite);

printf("\nСообщения, полученные родительским процессом из
канала:\n");

while (1) {

```

```

// читаем данные из пайпа
bzero(buf, 100);

int n = (int) read(descriptors[0], buf, 100);

// если пайп опустел заканчиваем работу
if (!n) {
    break;
}

// выводим сообщение из пайпа
printf("%s\n", buf);

// записываем в файл
fwrite(buf, sizeof(char), n, f);
}

// закрываем файл
fclose(f);

// закрываем пайп
close(descriptors[0]);

return 0;
}

```

Содержимое файла readFile.txt: It's message!

Содержимое файла writeFile.txt: (пусто)

Вывод программы:

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/8$ gcc 8.c -o 8
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/8$ ./8

```


Создан неименованный канал

Дочерний процесс читает данные из файла readFile.txt и записывает их в канал

Родительский процесс читает данные из канала и записывает их в файл writeFile.txt

Сообщения, полученные родительским процессом из канала:

It's message!

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/8\$

Содержимое файла readFile.txt: It's message!

Содержимое файла writeFile.txt: It's message!

Лог отражает корректную работу программы и передачу данных между родственными процессами через неименованный канал.

9. Задание: Реализуйте канал fifo. Приведите в отчете фрагмент файла, содержащего ограничения для IPC (для каналов и сообщений).

Была написаны 2 программы:

1. Программа-сервер создает 2 канала, открывает первый из них для записи и записывает туда имя файла, из которого программа-клиент должна прочесть данные. Программа также открывает второй канал для чтения, из которого считывает переданные клиентом данные. В конце своей работы каналы закрываются.
2. Программа-клиент открывает первый канал для чтения и читает оттуда имя файла для получения данных. Программа также открывает

второй канал для записи, в который записывает считанные из файла данные.

Исходный код: Файл server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>

#define DEF_FILENAME "buffer.txt"

int main(int argc, char **argv) {
    // имя файла для записи
    char fileName[30];

    // если не передан файл для записи
    if (argc < 2) {
        // используем файл по умолчанию
        printf("Используем файл по умолчанию '%s'\n",
DEF_FILENAME);
        strcpy(fileName, DEF_FILENAME);
    } else
        // в противном случае используем переданный файл
        strcpy(fileName, argv[1]);

    printf("Создание канала 1\n");

    // создаем первый именованный канал
    // устанавливаем для всех права на чтение и на запись
```

```

int res = mknod("channel1", S_IFIFO | 0666, 0);

// в случае ошибки
if (res) {

    printf("Ошибка создания канала 1\n");

    // удаляем первый канал
    unlink("channel1");

    // удаляем первый канал
    unlink("channel2");

    exit(1);

}

printf("Создание канала 2\n");

// создаем второй именнованный канал
// устанавливаем для всех права на чтение и на запись
res = mknod("channel2", S_IFIFO | 0666, 0);

// в случае ошибки
if (res) {

    printf("Ошибка создания канала 2\n");

    // удаляем первый канал
    unlink("channel1");

    // удаляем первый канал
    unlink("channel2");

    exit(1);

}

printf("Открываем канал 1 для записи\n");

// открываем первый канал для записи
int chan1 = open("channel1", O_WRONLY);

// в случае ошибки

```

```
if (chan1 == -1) {

    printf("Ошибка открытия канала 1 для записи\n");

    // удаляем первый канал

    unlink("channel1");

    // удаляем первый канал

    unlink("channel2");

    exit(1);

}

printf("Открываем канал 2 для чтения\n");

// открываем второй канал для чтения

int chan2 = open("channel2", O_RDONLY);

// в случае ошибки

if (chan2 == -1) {

    printf("Ошибка открытия канала 2 для чтения\n");

    // удаляем первый канал для записи

    unlink("channel1");

    // удаляем первый канал

    unlink("channel2");

    exit(1);

}


printf("Записываем имя файла в канал 1\n");

// пишем имя файла в первый канал для записи

write(chan1, fileName, strlen(fileName));


// читаем содержимое файла из второго канала для чтения

char buf[100];
```

```
printf("Ожидаем, пока клиент что-нибудь запишет в канал  
2\n");

printf("Нажмите Enter для отображения считанного из канала 2  
сообщения\n");

// ждем ввода пользователя
getchar();

// принимаем сообщение
for (;;) {

    // очищаем буфер
    bzero(buf, 100);

    // читаем данные из канала для чтения
    res = (int) read(chan2, buf, 100);

    // в случае ошибки
    if (res <= 0)

        // заканчиваем работу
        break;

    // выводим прочитанное содержимое
    printf("Принятое сообщение из канала 2:\n%s", buf);
}

printf("\nЗакрываем и удаляем каналы\n");

// закрываем первый канал
close(chan1);

// закрываем второй канал
close(chan2);

// удаляем первый канал
unlink("channel1");

// удаляем первый канал
unlink("channel2");
```

```
    printf("Работа сервера завершена\n");  
    return 0;  
}
```

Файл client.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <string.h>  
#include <fcntl.h>  
  
int main() {  
    printf("Открываем канал 1 для чтения\n");  
    // открываем первый канал для чтения  
    int chan1 = open("channel1", O_RDONLY);  
    // в случае ошибки  
    if (chan1 == -1) {  
        printf("Ошибка открытия канала 1 для чтения\n");  
        // удаляем первый канал  
        unlink("channel1");  
        // удаляем первый канал  
        unlink("channel2");  
        exit(1);  
    }  
    printf("Открываем канал 2 для записи\n");  
    // открываем второй канал для чтения  
    int chan2 = open("channel2", O_WRONLY);
```

```
// в случае ошибки

if (chan2 == -1) {

    printf("Ошибка открытия канала 2 для записи\n");

    // удаляем первый канал

    unlink("channel1");

    // удаляем первый канал

    unlink("channel2");

    exit(1);

}


// имя файла для чтения

char fileName[100];

// очищаем буфер

bzero(fileName, 100);


printf("Читаем имя файла из канала 1\n");

// читаем имя файла из первого канала для чтения

int res = (int) read(chan1, fileName, 100);

// в случае ошибки

if (res <= 0) {

    printf("Ошибка чтения имени файла из канала 1\n");

    // удаляем первый канал

    unlink("channel1");

    // удаляем первый канал

    unlink("channel2");

    exit(1);

}
```

```
printf("Открываем файл %s\n", fileName);

// открываем файл на чтение
FILE *f = fopen(fileName, "r");

// в случае ошибки
if (!f) {

    printf("Ошибка открытия файла %s\n", fileName);

    // удаляем первый канал
    unlink("channel1");

    // удаляем первый канал
    unlink("channel2");

    exit(1);

}

printf("Читаем данные из файла и пишем в канал 2\n");

// читаем из файла и пишем во второй канал
char buf[100];

// пока файл не пуст
while (!feof(f)) {

    // читаем данные из файла
    res = (int) fread(buf, sizeof(char), 100, f);

    // пишем их в канал 2
    write(chan2, buf, res);

}

// закрываем файл
fclose(f);

printf("Закрываем каналы\n");

// закрываем первый канал
close(chan1);
```



```
// закрываем второй канал  
close(chan2);  
  
printf("Работа клиента завершена\n");  
return 0;  
}
```

Вывод программы:

В первом терминале, где запущен сервер:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/9$ gcc server.c  
-o s
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/9$ ./s
```

Используем файл по умолчанию 'buffer.txt'

Создание канала 1

Создание канала 2

Открываем канал 1 для записи

Открываем канал 2 для чтения

Записываем имя файла в канал 1

Ожидаем, пока клиент что-нибудь запишет в канал 2

Нажмите Enter для отображения считанного из канала 2 сообщения

Принятое сообщение из канала 2:

It's buffer!

Hi!

Закрываем и удаляем каналы

Работа сервера завершена

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/9$
```

Во втором терминале, где запущен клиент:

```
        if (events[i].data.fd == chan1 && events[i].events &
EPOLLHUP && !epollhup) {
            epollhup = 1;
            printf("EPOLLHUP: канал 1 закрылся, клиент 1 закончил
свою работу\n");
            // задаем сообщению тип 3
            mes.type = 3L;
            // очищение буфера
            bzero(mes.buf, 100);
            // запись сообщения
            sprintf(mes.buf, "Клиент 1 закончил свою работу");
            // дописываем символ конца строки
            mes.buf[strlen(mes.buf)] = '\0';
            // отправляем это сообщение
            res = msgsnd(queue, (void *) &mes, sizeof(Message), 0);
            // в случае ошибки
            if (res != 0) {
                printf("Ошибка отправки сообщения\n");
                unlink("channel1");
                close(chan1);
                close(epoll_fd);
                exit(1);
            }
            // если произошло событие EPOLLIN
        } else if (events[i].data.fd == chan1 && events[i].events
& EPOLLIN && !epollin) {
            epollin = 1;
            printf("EPOLLIN: канал 1 открылся, подключился клиент
1\n");

            // задаем сообщению тип 2
            mes.type = 2L;
            // очищение буфера
            bzero(mes.buf, 100);
            // запись сообщения
            sprintf(mes.buf, "Клиент 1 подключился");
            // дописываем символ конца строки
            mes.buf[strlen(mes.buf)] = '\0';
```

```

        // отправляем это сообщение
        res = msgsnd(queue, (void *) &mes, sizeof(Message), 0);
        // в случае ошибки
        if (res != 0) {
            printf("Ошибка отправки сообщения\n");
            unlink("channel1");
            close(chan1);
            close(epoll_fd);
            exit(1);
        }
    }
}

close(epoll_fd);
return 0;
}

```

В терминале сервера:

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$ gcc server.c -
o s
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$ gcc client1.c
-o c1
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$ gcc client2.c
-o c2
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$ ./s
Используем файл с ключом по умолчанию key.txt
Сообщение клиента типа 2: Клиент 1 подключился
Сообщение клиента типа 1: message
Сообщение клиента типа 1: hi
Сообщение клиента типа 3: Клиент 1 закончил свою работу
^C
Работа сервера завершена
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$

```

В терминале клиента 2:

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$ ./c2
Используем файл с ключом по умолчанию key.txt
Создание канала 1 для индикации работы клиента 1

```

```
EPOLLIN: канал 1 открылся, подключился клиент 1
EPOLLHUP: канал 1 закрылся, клиент 1 закончил свою работу
^C
Работа клиента 2 завершена
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$
```

В терминале клиента 1:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$ ./c1
Используем файл с ключом по умолчанию key.txt
Записываем в канал 1 "client1"
message
hi
^C
Работа клиента 1 завершена
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR5/11$
```

Как видно из логов, клиент 2 успешно отправляет сообщения определенного типа в зависимости от случившегося события. Сообщения клиента 1 при этом имеют свой, отличный тип.

12. **Задание:** Выполните передачу информации локально посредством сокетов по TCP/IP.

Решение:

Были написаны 2 программы, которые вместе образуют эхо-сервер:

1. Программа-сервер прослушивает порт, при подключении клиента создает поток для его обработки. При отправке сообщения клиентом оно считывается сервером, выводится на экран и отправляется обратно клиенту как подтверждение успешного обмена данными.

2. Программа-клиент устанавливает соединение с сервером, считывает ввод пользователя и отправляет его серверу. при введении пустой строки клиент завершает свою работу.

Для передачи данных используются TCP-сокеты (SOCK_STREAM).

Для обмена данными используются 2 вспомогательные функции recvFix и sendFix.

Исходный код: Файл server.c

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <arpa/inet.h>
#include <unistd.h>

#define DEF_PORT 8888

// Функция получения сообщения
int readFix(int sock, char *buf, int bufSize, int flags) {
    // Длина сообщения
    size_t msgLength = 0;
    // Получение длины сообщения
    int res = (int) recv(sock, &msgLength, sizeof(size_t), flags
| MSG_WAITALL);
    // Если не удалось получить, то вернуть код
    if (res <= 0) {
        return res;
    }
    // Если длина сообщения больше размера буфера
    if (res > bufSize) {
        // Вывод сообщения об ошибке и завершение программы
        printf("Ошибка: длина сообщения больше размера буфера\n");
        exit(1);
    }
}
```

```

    // Получение сообщения и возврат полученного значения
    return (int) recv(sock, buf, msgLength, flags | MSG_WAITALL);
}

// Функция отправки данных
int sendFix(int sock, char *buf, int flags) {
    // Длина сообщения
    size_t msgLength = strlen(buf) + 1;
    // Отправка длины сообщения
    int res = (int) send(sock, &msgLength, sizeof(size_t),
flags);
    // Если не удалось отправить, то вернуть результат
    if (res <= 0) {
        return res;
    }
    // Отправить само сообщение и вернуть результат
    return (int) send(sock, buf, msgLength, flags);
}

// Обработчик клиента
void *clientHandler(void *args) {
    // Приведения аргумента к номеру сокета
    int sock = (int) args;
    // Буфер для сообщения
    char buf[100];
    // Результат
    int res;

    // Основной цикл работы
    while (1) {
        // Очистка буфера
        bzero(buf, 100);
        // Получение сообщения
        res = readFix(sock, buf, 100, 0);
        // В случае ошибки
        if (res <= 0) {
            perror("Ошибка получения данных клиента");
            pthread_exit(NULL);
            exit(0);
        }
        // Вывод сообщения

```

```

        printf("Клиент отправил: %s\n", buf);
        // Отправка сообщения обратно клиенту
        res = sendFix(sock, buf, 0);
        // В случае ошибки
        if (res <= 0) {
            perror("Ошибка отправки данных");
            pthread_exit(NULL);
        }
    }
}

int main(int argc, char **argv) {
    int port;
    if (argc < 2) {
        printf("Используем порт по умолчанию: %d\n", DEF_PORT);
        port = DEF_PORT;
    } else {
        port = atoi(argv[2]);
    }

    // Инициализация структуры заданным адресом прослушивателя
    входящих соединений
    struct sockaddr_in listenerInfo;
    listenerInfo.sin_family = AF_INET;
    listenerInfo.sin_port = htons(port);
    listenerInfo.sin_addr.s_addr = htonl(INADDR_ANY);

    // Вывод адреса в строковом виде
    struct in_addr display = {INADDR_ANY};
    printf("\nАдрес сокета %s:%d (%s:%d)\n",
        inet_ntoa(listenerInfo.sin_addr), listenerInfo.sin_port,
        inet_ntoa(display), port);

    // Инициализация сокета
    int listener = socket(AF_INET, SOCK_STREAM, 0);
    // В случае ошибки
    if (listener < 0) {
        perror("Ошибка создания сокета");
        exit(1);
    }
}

```

```

// Привязка сокета к адресу
int res = bind(listener, (struct sockaddr *) &listenerInfo,
sizeof(listenerInfo));
// В случае ошибки
if (res < 0) {
    perror("Ошибка привязки сокета");
    close(listener);
    exit(1);
}
// Включение режима прослушивания
res = listen(listener, 5);
// В случае ошибки
if (res) {
    perror("Ошибка прослушивания");
    close(listener);
    exit(1);
}

// основной цикл работы
while (1) {
    // Принятие входящего соединения и создание сокета
    int client = accept(listener, NULL, NULL);
    // Инициализация потока для обработки сокета
    pthread_t thread;
    res = pthread_create(&thread, NULL, clientHandler, (void *)
(client));
    // В случае ошибки
    if (res) {
        printf("Ошибка создания потока\n");
        close(listener);
        pthread_exit(NULL);
    }
}
return 0;
}

```

Файл client.c

```

#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>

```



```

#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define DEF_PORT 8888
#define DEF_IP "127.0.0.1"

// Функция получения сообщения
int readFix(int sock, char *buf, int bufSize, int flags) {
    // Длина сообщения
    size_t msgLength = 0;
    // Получение длины сообщения
    int res = (int) recv(sock, &msgLength, sizeof(size_t), flags
| MSG_WAITALL);
    // Если не удалось получить, то вернуть код
    if (res <= 0)
        return res;
    // Если длина сообщения больше размера буфера
    if (res > bufSize) {
        // Вывод сообщения об ошибке и завершение программы
        printf("Ошибка: длина сообщения больше размера буфера\n");
        exit(1);
    }
    // Получение сообщения и возврат полученного значения
    return (int) recv(sock, buf, msgLength, flags | MSG_WAITALL);
}

// Функция отправки данных
int sendFix(int sock, char *buf, int flags) {
    // Длина сообщения
    size_t msgLength = strlen(buf) + 1;
    // Отправка длины сообщения
    int res = (int) send(sock, &msgLength, sizeof(size_t),
flags);
    // Если не удалось отправить, то вернуть результат
    if (res <= 0)
        return res;
    // Отправить само сообщение и вернуть результат
    return (int) send(sock, buf, msgLength, flags);
}

```

```

int main(int argc, char **argv) {
    char *addr;
    int port;
    if (argc < 3) {
        printf("Используем порт по умолчанию: %d\n", DEF_PORT);
        port = DEF_PORT;
    } else {
        port = atoi(argv[2]);
    }

    if (argc < 2) {
        printf("Используем адрес по умолчанию: %s\n", DEF_IP);
        addr = DEF_IP;
    } else {
        addr = argv[1];
    }

    // Создание сокета
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("Ошибка создания сокета\n");
        exit(1);
    }

    struct sockaddr_in peer;
    peer.sin_family = AF_INET;
    peer.sin_port = htons(port);
    peer.sin_addr.s_addr = inet_addr(addr);

    // Вывод адреса в строковом виде
    struct in_addr display = {INADDR_LOOPBACK};
    printf("\nАдрес сокета %s:%d (%s:%d)\n",
        inet_ntoa(peer.sin_addr), peer.sin_port,
        addr, port);

    // Присоединяемся к серверу
    int res = connect(sock, (struct sockaddr *) &peer,
sizeof(peer));
    // В случае ошибки
    if (res) {
        perror("Ошибка подключения к серверу");
    }
}

```

```

    close(sock);
    exit(1);
}

// Буфер под сообщение
char buf[100];
// Основной цикл работы
while (1) {
    printf("\nВведите сообщение\n");
    // Очищение буфера
    bzero(buf, 100);
    // Считывание ввода
    fgets(buf, 100, stdin);
    // Добавление символа конца строки
    buf[strlen(buf) - 1] = '\0';
    // Если пользователь ничего не ввел
    if (strlen(buf) == 0) {
        // Завершаем работу
        printf("Конец работы клиента\n");
        return 0;
    }
    // Отправляем сообщение серверу
    res = sendFix(sock, buf, 0);
    // В случае ошибки
    if (res <= 0) {
        perror("Ошибка отправки сообщения серверу");
        close(sock);
        exit(1);
    }
    // Очищение буфера
    bzero(buf, 100);
    // Прием ответа сервера
    res = readFix(sock, buf, 100, 0);
    // В случае ошибки
    if (res <= 0) {
        perror("Ошибка получения сообщения от сервера");
        close(sock);
        exit(1);
    }
    printf("Ответ сервера: %s\n", buf);
}

```

```
    return 0;
}
```

Вывод программы:

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ gcc
server.c -o s
server.c: In function 'clientHandler':
server.c:49:16: warning: cast from pointer to integer of
different size [-Wpointer-to-int-cast]
    49 |         int sock = (int) args;
        |                        ^
server.c: In function 'main':
server.c:130:60: warning: cast to pointer from integer of
different size [-Wint-to-pointer-cast]
    130 |         res = pthread_create(&thread, NULL,
clientHandler, (void *) (client));
        |
```

^

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ ./s
Используем порт по умолчанию: 8888
```

Адрес сокета 0.0.0.0:47138 (0.0.0.0:8888)

```
Клиент отправил: Hi!
Клиент отправил: It's client!
Клиент отправил: Bye!
Ошибка получения данных клиента: Success
^C
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$
```

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ gcc
client.c -o c
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ ./c
Используем порт по умолчанию: 8888
Используем адрес по умолчанию: 127.0.0.1
```

```
Адрес сокета 127.0.0.1:47138 (127.0.0.1:8888)
```

```
Введите сообщение
```

```
Hi!
```

```
Ответ сервера: Hi!
```

```
Введите сообщение
```

```
It's client!
```

```
Ответ сервера: It's client!
```

```
Введите сообщение
```

```
Bye!
```

```
Ответ сервера: Bye!
```

```
Введите сообщение
```

```
Конец работы клиента
```

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$
```

Как видно из лога, клиент и сервер успешно обмениваются сообщениями локально (IP сервера - 0.0.0.0, а клиента - 127.0.0.1).

13. **Задание:** а затем в сетевом режиме (посредством сокетов по TCP/IP)

Решение:

Запустим описанный в задании 12 эхо-сервер в сетевом режиме: для этого сначала с помощью утилиты `ifconfig` определим IP-адрес сети.

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ ifconfig
enp88s0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
    ether 84:a9:38:8c:cd:11  txqueuelen 1000  (Ethernet)
    RX packets 0  bytes 0 (0.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 0  bytes 0 (0.0 B)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
enx00e04c68010e:
flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.105 netmask 255.255.255.0 broadcast
192.168.0.255
    inet6 fe80::c8dd:e7b:2cf9:390c prefixlen 64 scopeid
0x20<link>
    ether 00:e0:4c:68:01:0e txqueuelen 1000 (Ethernet)
    RX packets 240231 bytes 306960214 (306.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 93520 bytes 13323988 (13.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3762 bytes 361093 (361.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3762 bytes 361093 (361.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp0s20f3: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether f4:46:37:62:5b:6a txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Теперь при запуске клиента укажем адрес сети 192.168.0.105:

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ ./s
Используем порт по умолчанию: 8888
```

Адрес сокета 0.0.0.0:47138 (0.0.0.0:8888)

Клиент отправил: Hello!

Клиент отправил: Meow

Ошибка получения данных клиента: Success

^C

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$
```

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$ ./c  
192.168.0.105
```

Используем порт по умолчанию: 8888

Адрес сокета 192.168.0.105:47138 (192.168.0.105:8888)

Введите сообщение

Hello!

Ответ сервера: Hello!

Введите сообщение

Meow

Ответ сервера: Meow

Введите сообщение

Конец работы клиента

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/12$
```

Как видно из логово, эхо-сервер успешно выполняет обмен сообщениями по сети.

14. **Задание:** Организуйте взаимодействие с 10, 100 и 1000 клиентами в клиент-серверном приложении (посредством сокетов).
Оцените ограничения.

Решение: Программы, описанные в задании 12 были модифицированы следующим образом: программа-клиент создает нужное количество потоков-клиентов, и в каждом из них отправляет по 100 сообщений. Программа-сервер при обработке потока-клиента выводит тех клиентов, которые успешно передали все 100 сообщений, а по сигналу прерывания, перед завершением работы, список тех клиентов, которые

передали меньше 100 пакетов данных. Для передачи данных используется протокол TCP.

Исходный код: Файл server.c

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <signal.h>
#include <unistd.h>

#define DEF_PORT 8888
#define DEF_IP "127.0.0.1"

// Массив числа принятых пакетов каждым сокетом
volatile long long counts[1000];
// Количество сокетов
volatile int socketCount = 0;

// Функция получения сообщения
int readFix(int sock, char *buf, int bufSize, int flags) {
    // Длина сообщения
    size_t msgLength = 0;
    // Получение длины сообщения
    int res = (int) recv(sock, &msgLength, sizeof(size_t), flags
| MSG_WAITALL);
    // Если не удалось получить, то вернуть код
    if (res <= 0)
        return res;
    // Если длина сообщения больше размера буфера
    if (res > bufSize) {
        // Вывод сообщения об ошибке и завершение программы
        printf("Ошибка: длина сообщения больше размера буфера\n");
        exit(1);
    }
    // Получение сообщения и возврат полученного значения
```



```

    return (int) recv(sock, buf, msgLength, flags | MSG_WAITALL);
}

// Функция отправки данных
int sendFix(int sock, char *buf, int flags) {
    // Длина сообщения
    size_t msgLength = strlen(buf) + 1;
    // Отправка длины сообщения
    int res = (int) send(sock, &msgLength, sizeof(unsigned),
flags);
    // Если не удалось отправить, то вернуть результат
    if (res <= 0)
        return res;
    // Отправить само сообщение и вернуть результат
    return (int) send(sock, buf, msgLength, flags);
}

// Обработчик клиента
void *clientHandler(void *args) {
    // Увеличение числа сокетов
    socketCount++;
    // Приведения аргумента к номеру сокета
    int sock = (int) args;
    // Буфер для сообщения
    char buf[100];
    // Результат
    int res;

    // Основной цикл работы
    while (1) {
        // Очистка буфера
        bzero(buf, 100);
        // Получение пакета
        res = readFix(sock, buf, 100, 0);
        // В случае ошибки - выход из потока
        if (res <= 0) {
            perror("Ошибка получения данных клиента");
            pthread_exit(NULL);
        }
        // Увеличение количества принятых пакетов
        counts[sock - 4]++;
    }
}

```

```

        // При получении 100 пакетов - вывод сообщения
        if (counts[sock - 4] == 100) {
            printf("Клиент %d передал 100 пакетов данных\n", sock -
3);
        }
    }
}

// Обработчик прерывания
void sigintHandler() {
    // Вывод количества принятых пакетов каждым сокетом
    printf("\nКаждый из %d клиентов передал 100 пакетов данных,
кроме:\n", socketCount);
    // Проверяем все переданные сокеты
    for (int n = 0; n < socketCount; n++) {
        if (counts[n] != 100) {
            printf("Клиента %d - передано %lld пакетов данных\n", n +
1, counts[n]);
        }
    }
    // Сброс буфера
    fflush(stdout);
    // Установка диспозиции по умолчанию
    signal(SIGINT, SIG_DFL);
}

int main(int argc, char **argv) {
    int port;
    if (argc < 2) {
        printf("Используем порт по умолчанию: %d\n\n", DEF_PORT);
        port = DEF_PORT;
    } else {
        port = atoi(argv[2]);
    }

    // Установка обработчика сигнала
    signal(SIGINT, sigintHandler);

    // Инициализация структуры адреса прослушивателя входящих
соединений
    struct sockaddr_in listenerInfo;

```

```

listenerInfo.sin_family = AF_INET;
listenerInfo.sin_port = htons(port);
listenerInfo.sin_addr.s_addr = htonl(INADDR_ANY);

// Инициализация сокета
int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);
// Выход в случае ошибки
if (listener < 0) {
    perror("Ошибка создания сокета");
    exit(1);
}
// Привязка сокета к адресу
int res = bind(listener, (struct sockaddr *) &listenerInfo,
sizeof(listenerInfo));
// Выход в случае ошибки
if (res < 0) {
    perror("Ошибка привязки сокета");
    close(listener);
    exit(1);
}
// Включение режима прослушивания
res = listen(listener, 10);
// Выход в случае ошибки
if (res) {
    perror("Ошибка прослушивания");
    close(listener);
    exit(1);
}

// Основной цикл работы
while (1) {
    // Принятие входящего соединения и создание сокета
    int client = accept(listener, NULL, NULL);
    // Инициализация потока для обработки сокета
    pthread_t thread;
    res = pthread_create(&thread, NULL, clientHandler, (void *)
(client));
    // Вывод сообщения об ошибке
    if (res) {
        perror("Ошибка создания потока");
        close(listener);
    }
}

```

```

        pthread_exit(NULL);
    }
}
return 0;
}

```

Файл client.c

```

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

#define DEF_PORT 8888
#define DEF_IP "127.0.0.1"
#define DEF_THREADS_AMT 100

pthread_mutex_t mutex;
char *addr;
int port;

// Функция получения сообщения
int readFix(int sock, char *buf, int bufSize, int flags) {
    // Длина сообщения
    size_t msgLength = 0;
    // Получение длины сообщения
    int res = (int) recv(sock, &msgLength, sizeof(size_t), flags
| MSG_WAITALL);
    // Если не удалось получить, то вернуть код
    if (res <= 0)
        return res;
    // Если длина сообщения больше размера буфера
    if (res > bufSize) {
        // Вывод сообщения об ошибке и завершение программы
        printf("Ошибка: длина сообщения больше размера буфера\n");
        exit(1);
    }
}

```

```

    // Получение сообщения и возврат полученного значения
    return (int) recv(sock, buf, msgLength, flags | MSG_WAITALL);
}

// Функция отправки данных
int sendFix(int sock, char *buf, int flags) {
    // Длина сообщения
    size_t msgLength = strlen(buf) + 1;
    // Отправка длины сообщения
    int res = (int) send(sock, &msgLength, sizeof(size_t),
flags);
    // Если не удалось отправить, то вернуть результат
    if (res <= 0)
        return res;
    // Отправить само сообщение и вернуть результат
    return (int) send(sock, buf, msgLength, flags);
}

// Рабочий поток
void *threadWorker(void *arg) {
    // Захват мьютекса
    pthread_mutex_lock(&mutex);
    // Получение номера потока из аргументов
    int id = (int) arg;

    // Инициализация структуры интернет адреса
    struct sockaddr_in peer;
    peer.sin_family = AF_INET;
    peer.sin_port = htons(port);
    peer.sin_addr.s_addr = inet_addr(addr);

    // Инициализация сокета
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    // Выход в случае ошибки
    if (sock < 0) {
        perror("Ошибка создания сокета\n");
        exit(1);
    }

    // Подключение к сокету

```

```

        int res = connect(sock, (struct sockaddr *) &peer,
sizeof(peer));
        // В случае ошибки
        if (res) {
            perror("Ошибка подключения к серверу");
            close(sock);
            exit(1);
        }

        // Освобождение мьютекса
        pthread_mutex_unlock(&mutex);

        // Инициализация буфера
        char buf[100];
        sprintf(buf, "Поток %d", id);
        // Основной цикл работы - 100 итераций
        for (int i = 0; i < 100; i++) {
            // Отправка сообщения
            res = sendFix(sock, buf, 0);
            // Выход в случае ошибки
            if (res <= 0) {
                fprintf(stderr, "Поток %d: Ошибка отправки сообщения
серверу", id);
                perror("");
                exit(1);
            }
        }
    }
}

```

```

int main(int argc, char **argv) {
    int threadAmt;
    // Проверка числа аргументов
    if (argc < 4) {
        printf("Используем порт по умолчанию: %d\n", DEF_PORT);
        port = DEF_PORT;
    } else {
        port = atoi(argv[3]);
    }
    if (argc < 3) {
        printf("Используем адрес по умолчанию: %s\n", DEF_IP);
        addr = DEF_IP;
    }
}

```

```

    } else {
        addr = argv[2];
    } if (argc < 2) {
        printf("Используем количество потоков по умолчанию: %d\n",
DEF_THREADS_AMT);
        threadAmt = DEF_THREADS_AMT;
    } else {
        threadAmt = atoi(argv[1]);
    }

    // Инициализация потоков
    for (int i = 0; i < threadAmt; i++) {
        pthread_t thread;
        pthread_create(&thread, NULL, threadWorker, (void *) i);
    }

    printf("\nСоздано %d потоков-клиентов\n", threadAmt);
    // Приостановка главного потока на 40 секунд
    sleep(20);
    printf("Конец работы программы\n");
    return 0;
}

```

Вывод программы:

10 клиентов

В терминале клиента:

```

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ gcc
client.c -o c
<...>
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ ./c 10
Используем порт по умолчанию: 8888
Используем адрес по умолчанию: 127.0.0.1

Создано 10 потоков-клиентов
Конец работы программы
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$

```

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ gcc
server.c -o s
<...>
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ ./s
Используем порт по умолчанию: 8888
```

```
Клиент 1 передал 100 пакетов данных
Клиент 3 передал 100 пакетов данных
Клиент 4 передал 100 пакетов данных
Клиент 2 передал 100 пакетов данных
Клиент 6 передал 100 пакетов данных
Клиент 5 передал 100 пакетов данных
Клиент 7 передал 100 пакетов данных
Клиент 8 передал 100 пакетов данных
Клиент 9 передал 100 пакетов данных
Клиент 10 передал 100 пакетов данных
```

```
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
```

^C

Каждый из 10 клиентов передал 100 пакетов данных, кроме:

^C

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$
```

100 клиентов

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ ./c
Используем порт по умолчанию: 8888
Используем адрес по умолчанию: 127.0.0.1
Используем количество потоков по умолчанию: 100
```

Создано 100 потоков-клиентов

Конец работы программы

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14\$

В терминале сервера:

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14\$./s

Используем порт по умолчанию: 8888

Клиент 1 передал 100 пакетов данных
Клиент 2 передал 100 пакетов данных
Клиент 3 передал 100 пакетов данных
Клиент 4 передал 100 пакетов данных
Клиент 5 передал 100 пакетов данных
Клиент 6 передал 100 пакетов данных
Клиент 7 передал 100 пакетов данных
Клиент 8 передал 100 пакетов данных
Клиент 9 передал 100 пакетов данных
Клиент 11 передал 100 пакетов данных
Клиент 10 передал 100 пакетов данных
Клиент 12 передал 100 пакетов данных
Клиент 13 передал 100 пакетов данных
Клиент 14 передал 100 пакетов данных
Клиент 15 передал 100 пакетов данных
Клиент 17 передал 100 пакетов данных
Клиент 19 передал 100 пакетов данных
Клиент 21 передал 100 пакетов данных
Клиент 20 передал 100 пакетов данных
Клиент 23 передал 100 пакетов данных
Клиент 22 передал 100 пакетов данных
Клиент 16 передал 100 пакетов данных
Клиент 24 передал 100 пакетов данных
Клиент 18 передал 100 пакетов данных
Клиент 27 передал 100 пакетов данных
Клиент 25 передал 100 пакетов данных
Клиент 26 передал 100 пакетов данных
Клиент 28 передал 100 пакетов данных
Клиент 29 передал 100 пакетов данных
Клиент 30 передал 100 пакетов данных
Клиент 31 передал 100 пакетов данных
Клиент 32 передал 100 пакетов данных
Клиент 34 передал 100 пакетов данных
Клиент 33 передал 100 пакетов данных
Клиент 35 передал 100 пакетов данных
Клиент 36 передал 100 пакетов данных

[illegible]

```
Клиент 82 передал 100 пакетов данных
Клиент 83 передал 100 пакетов данных
Клиент 84 передал 100 пакетов данных
Клиент 85 передал 100 пакетов данных
Клиент 86 передал 100 пакетов данных
Клиент 87 передал 100 пакетов данных
Клиент 88 передал 100 пакетов данных
Клиент 89 передал 100 пакетов данных
Клиент 90 передал 100 пакетов данных
Клиент 91 передал 100 пакетов данных
Клиент 92 передал 100 пакетов данных
Клиент 93 передал 100 пакетов данных
Клиент 95 передал 100 пакетов данных
Клиент 94 передал 100 пакетов данных
Клиент 97 передал 100 пакетов данных
Клиент 96 передал 100 пакетов данных
Клиент 98 передал 100 пакетов данных
Клиент 100 передал 100 пакетов данных
Клиент 99 передал 100 пакетов данных
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
<...>
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
^C
Каждый из 100 клиентов передал 100 пакетов данных, кроме:
^C
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$
```

1000 клиентов

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ ./с 1000
Используем порт по умолчанию: 8888
Используем адрес по умолчанию: 127.0.0.1
```

Создано 1000 потоков-клиентов

Конец работы программы

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$
```

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$ ./s
Используем порт по умолчанию: 8888
```

```
Клиент 1 передал 100 пакетов данных
Клиент 2 передал 100 пакетов данных
<...>
Клиент 999 передал 100 пакетов данных
Клиент 1000 передал 100 пакетов данных
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
<...>
Ошибка получения данных клиента: Success
Ошибка получения данных клиента: Success
^C
Каждый из 1000 клиентов передал 100 пакетов данных, кроме:
^C
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/14$
```

Как видно из логов, взаимодействие сервера с 10, 100 и 100 клиентов проходит успешно. Отметим некоторые особенности работы TCP-протокола:

- Все клиенты успешно передали все свои данные без потерь;
- Клиенты передавали свои данные достаточно последовательно, в порядке создания.

Небольшие “сбивки”, когда данные сначала отправил клиент 3, а потом 2, я думаю, можно объяснить тем, что при отправке данных клиентом 2 произошла ошибка, приведшая к потере пакета, и клиент был вынужден переотправить данные заново, в связи с чем обработка клиента 2 затянулась и закончилась позднее обработки клиента 3.

- Для получения сообщения между сервером и клиентом устанавливается соединение с помощью функции `accept()`.

Можно сделать вывод, что TCP-протокол является надежным методом передачи данных.

15. **Задание:** Выполните аналогичное взаимодействие на основе UDP,

Решение: Была написана программа аналогичная заданию 14, осуществляющая взаимодействие с 10, 100 и 1000 клиентами с помощью UDP-протокола.

Программа-клиент так же, как и в предыдущем задании, создает нужное число клиентов-потоков, каждый из которых отправляет по 100 сообщений серверу.

Программа-сервер сохраняет количество принятых от каждого клиента сообщений в массив, и раз в 100 принятых сообщений выводит номер текущего клиента. При получении сигнала прерывания, сервер выводит количество переданных каждым из клиентов сообщений, общее число полученных сообщений и завершает работу.

Для передачи данных используется протокол UDP.

Исходный код: Файл `server.c`

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <unistd.h>
#include <signal.h>

#define DEF_PORT 8888
#define DEF_IP "127.0.0.1"
#define DEF_CLIENTS_AMT 1000

// Массив числа принятых пакетов каждым сокетом
volatile long long *counts;
// Количество сокетов
volatile int totalSocketCount = 0;

// Обработчик прерывания
void sigHandler() {
    printf("\n");
    // Вывод общего количества пакетов, принятых каждым сокетом
    for (int i = 0; i < DEF_CLIENTS_AMT; ++i) {
        printf("Клиент %d передал %lld пакетов данных\n", i + 1,
counts[i]);
    }
    // Вывод общего количества принятых пакетов
    printf("\nОбщее количество принятых пакетов данных: %d из %d
(%d%%)\n", totalSocketCount, DEF_CLIENTS_AMT * 100,
totalSocketCount / DEF_CLIENTS_AMT);
    free((long long *) counts);
    // Восстановление обработчика
    signal(SIGINT, SIG_DFL);
}

int main(int argc, char **argv) {
    int port;
    if (argc < 2) {
        printf("Используем порт по умолчанию: %d\n\n", DEF_PORT);
        port = DEF_PORT;
    } else {
        port = atoi(argv[2]);
    }

    // Установка обработчика прерывания
    signal(SIGINT, sigHandler);

```

```

    counts = (volatile long long *) calloc(1000, sizeof(long
long));

    // Инициализация структуры адреса
    struct sockaddr_in listenerInfo;
    listenerInfo.sin_family = AF_INET;
    listenerInfo.sin_port = htons(port);
    listenerInfo.sin_addr.s_addr = htonl(INADDR_ANY);

    // Создание сокета
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    // Выход в случае ошибки
    if (sock < 0) {
        perror("Ошибка создания сокета");
        exit(1);
    }
    // Привязка сокета к адресу
    int res = bind(sock, (struct sockaddr *) &listenerInfo,
sizeof(listenerInfo));
    // Выход в случае ошибки
    if (res < 0) {
        perror("Ошибка привязки сокета");
        close(sock);
        exit(1);
    }

    // Буфер сообщения
    char buf[100];
    int prev_client = 1001;
    // Основной цикл работы
    while (1) {
        // Получение сообщения
        ssize_t n = recvfrom(sock, buf, sizeof buf, 0, NULL, NULL);
        // Вывод в случае ошибки
        if (n < 0) {
            perror("Ошибка получения сообщения");
        }
        // Выход в случае получения слишком большого числа данных
        if (n > sizeof buf) {

```

```

        printf("Ошибка: длина сообщения больше размера
буфера\n");
        exit(1);
    }
    // Увеличение общего количества принятых пакетов
    totalSocketCount++;
    // Увеличение общего количества пакетов, принятых данным
клиентом
    int ind = atoi(buf);
    counts[ind]++;
    if (totalSocketCount % 100 == 0){
        printf("Обработан клиент %d\n", ind + 1);
    }
}
return 0;
}

```

Файл client.c

```

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

#define DEF_PORT 8888
#define DEF_IP "127.0.0.1"
#define DEF_THREADS_AMT 100

pthread_mutex_t mutex;
char *addr;
int port;

// Рабочий поток
void *threadWorker(void *arg) {
    // Захват мьютекса
    pthread_mutex_lock(&mutex);
    // Получение номера потока из аргументов

```



```

int id = (int) arg;
// Инициализация структуры интернет адреса
struct sockaddr_in peer;
peer.sin_family = AF_INET;
peer.sin_port = htons(port);
peer.sin_addr.s_addr = inet_addr(addr);

// Инициализация сокета
int sock = socket(AF_INET, SOCK_DGRAM, 0);
// Выход в случае ошибки
if (sock < 0) {
    perror("Ошибка создания сокета");
    exit(1);
}

// Подключение к сокету
int res = connect(sock, (struct sockaddr *) &peer,
sizeof(peer));
// Выход в случае ошибки
if (res) {
    perror("Ошибка подключения к серверу");
    close(sock);
    exit(1);
}

// Освобождение мьютекса
pthread_mutex_unlock(&mutex);

// Инициализация буфера
char buf[100];
sprintf(buf, "%d", id);
// Основной цикл работы - 100 итераций
for (int i = 0; i < 100; i++) {
    // Отправка сообщения
    res = send(sock, buf, strlen(buf) + 1, 0);
    // Выход в случае ошибки
    if (res <= 0) {
        fprintf(stderr, "Поток %d: Ошибка отправки сообщения
серверу", id);
        perror("");
    }
}

```

```

        exit(1);
    }
}

int main(int argc, char **argv) {
    int threadAmt;
    // Проверка числа аргументов
    if (argc < 4) {
        printf("Используем порт по умолчанию: %d\n", DEF_PORT);
        port = DEF_PORT;
    } else {
        port = atoi(argv[3]);
    }

    if (argc < 3) {
        printf("Используем адрес по умолчанию: %s\n", DEF_IP);
        addr = DEF_IP;
    } else {
        addr = argv[2];
    }

    if (argc < 2) {
        printf("Используем количество потоков по умолчанию: %d\n",
DEF_THREADS_AMT);
        threadAmt = DEF_THREADS_AMT;
    } else {
        threadAmt = atoi(argv[1]);
    }

    // Инициализация потоков
    for (int i = 0; i < threadAmt; i++) {
        pthread_t thread;
        pthread_create(&thread, NULL, threadWorker, (void *) i);
    }

    printf("\nСоздано %d потоков-клиентов\n", threadAmt);
    // Приостановка главного потока на 40 секунд
    sleep(20);
    printf("Конец работы программы\n");
    return 0;
}

```

Вывод программы:

10 клиентов

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ gcc
client.c -o c
<...>
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ ./c 10
Используем порт по умолчанию: 8888
Используем адрес по умолчанию: 127.0.0.1
```

Создано 10 потоков-клиентов

Конец работы программы

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$
```

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ gcc
server.c -o s
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ ./s
Используем порт по умолчанию: 8888
```

Обработан клиент 2

Обработан клиент 1

Обработан клиент 2

Обработан клиент 3

Обработан клиент 5

Обработан клиент 6

^C

Клиент 1 передал 96 пакетов данных

Клиент 2 передал 67 пакетов данных

Клиент 3 передал 88 пакетов данных

Клиент 4 передал 71 пакетов данных

Клиент 5 передал 80 пакетов данных

Клиент 6 передал 56 пакетов данных

Клиент 7 передал 52 пакетов данных

Клиент 8 передал 63 пакетов данных

Клиент 9 передал 62 пакетов данных

Клиент 10 передал 64 пакетов данных

Общее количество принятых пакетов данных: 699 из 1000 (69%)

^C

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$
```

100 клиентов

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ ./с
```

Используем порт по умолчанию: 8888

Используем адрес по умолчанию: 127.0.0.1

Используем количество потоков по умолчанию: 100

Создано 100 потоков-клиентов

Конец работы программы

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$
```

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ ./s
```

Используем порт по умолчанию: 8888

Обработан клиент 4
Обработан клиент 7
Обработан клиент 7
Обработан клиент 10
Обработан клиент 8
Обработан клиент 15
Обработан клиент 18
Обработан клиент 13
Обработан клиент 13
Обработан клиент 27
Обработан клиент 17
Обработан клиент 24
Обработан клиент 29
Обработан клиент 31
Обработан клиент 21
Обработан клиент 25
Обработан клиент 20
Обработан клиент 19
Обработан клиент 34
Обработан клиент 41
Обработан клиент 49

Обработан клиент 46
Обработан клиент 42
Обработан клиент 45
Обработан клиент 54
Обработан клиент 60
Обработан клиент 53
Обработан клиент 69
Обработан клиент 56
Обработан клиент 55
Обработан клиент 70
Обработан клиент 72
Обработан клиент 75
Обработан клиент 76
Обработан клиент 79
Обработан клиент 80
Обработан клиент 83
Обработан клиент 79
Обработан клиент 94
Обработан клиент 93
Обработан клиент 93
Обработан клиент 88
Обработан клиент 88
Обработан клиент 99
Обработан клиент 98
Обработан клиент 89
Обработан клиент 100
Обработан клиент 100

^C

Клиент 1 передал 100 пакетов данных
Клиент 2 передал 99 пакетов данных
Клиент 3 передал 78 пакетов данных
Клиент 4 передал 71 пакетов данных
Клиент 5 передал 41 пакетов данных
Клиент 6 передал 51 пакетов данных
Клиент 7 передал 59 пакетов данных
Клиент 8 передал 59 пакетов данных
Клиент 9 передал 44 пакетов данных
Клиент 10 передал 52 пакетов данных
Клиент 11 передал 41 пакетов данных
Клиент 12 передал 37 пакетов данных
Клиент 13 передал 36 пакетов данных
Клиент 14 передал 31 пакетов данных
Клиент 15 передал 37 пакетов данных
Клиент 16 передал 39 пакетов данных
Клиент 17 передал 71 пакетов данных

Клиент	18	передал	34	пакетов	данных
Клиент	19	передал	43	пакетов	данных
Клиент	20	передал	51	пакетов	данных
Клиент	21	передал	36	пакетов	данных
Клиент	22	передал	68	пакетов	данных
Клиент	23	передал	44	пакетов	данных
Клиент	24	передал	81	пакетов	данных
Клиент	25	передал	46	пакетов	данных
Клиент	26	передал	54	пакетов	данных
Клиент	27	передал	70	пакетов	данных
Клиент	28	передал	72	пакетов	данных
Клиент	29	передал	57	пакетов	данных
Клиент	30	передал	60	пакетов	данных
Клиент	31	передал	66	пакетов	данных
Клиент	32	передал	35	пакетов	данных
Клиент	33	передал	33	пакетов	данных
Клиент	34	передал	33	пакетов	данных
Клиент	35	передал	33	пакетов	данных
Клиент	36	передал	32	пакетов	данных
Клиент	37	передал	33	пакетов	данных
Клиент	38	передал	32	пакетов	данных
Клиент	39	передал	37	пакетов	данных
Клиент	40	передал	28	пакетов	данных
Клиент	41	передал	32	пакетов	данных
Клиент	42	передал	34	пакетов	данных
Клиент	43	передал	31	пакетов	данных
Клиент	44	передал	31	пакетов	данных
Клиент	45	передал	28	пакетов	данных
Клиент	46	передал	29	пакетов	данных
Клиент	47	передал	28	пакетов	данных
Клиент	48	передал	30	пакетов	данных
Клиент	49	передал	29	пакетов	данных
Клиент	50	передал	27	пакетов	данных
Клиент	51	передал	29	пакетов	данных
Клиент	52	передал	30	пакетов	данных
Клиент	53	передал	38	пакетов	данных
Клиент	54	передал	41	пакетов	данных
Клиент	55	передал	51	пакетов	данных
Клиент	56	передал	46	пакетов	данных
Клиент	57	передал	66	пакетов	данных
Клиент	58	передал	35	пакетов	данных
Клиент	59	передал	43	пакетов	данных
Клиент	60	передал	33	пакетов	данных
Клиент	61	передал	28	пакетов	данных
Клиент	62	передал	28	пакетов	данных

Клиент 63 передал 28 пакетов данных
Клиент 64 передал 32 пакетов данных
Клиент 65 передал 30 пакетов данных
Клиент 66 передал 42 пакетов данных
Клиент 67 передал 54 пакетов данных
Клиент 68 передал 38 пакетов данных
Клиент 69 передал 61 пакетов данных
Клиент 70 передал 100 пакетов данных
Клиент 71 передал 50 пакетов данных
Клиент 72 передал 90 пакетов данных
Клиент 73 передал 85 пакетов данных
Клиент 74 передал 82 пакетов данных
Клиент 75 передал 79 пакетов данных
Клиент 76 передал 66 пакетов данных
Клиент 77 передал 55 пакетов данных
Клиент 78 передал 48 пакетов данных
Клиент 79 передал 49 пакетов данных
Клиент 80 передал 44 пакетов данных
Клиент 81 передал 44 пакетов данных
Клиент 82 передал 38 пакетов данных
Клиент 83 передал 33 пакетов данных
Клиент 84 передал 100 пакетов данных
Клиент 85 передал 99 пакетов данных
Клиент 86 передал 39 пакетов данных
Клиент 87 передал 40 пакетов данных
Клиент 88 передал 43 пакетов данных
Клиент 89 передал 99 пакетов данных
Клиент 90 передал 42 пакетов данных
Клиент 91 передал 34 пакетов данных
Клиент 92 передал 33 пакетов данных
Клиент 93 передал 33 пакетов данных
Клиент 94 передал 35 пакетов данных
Клиент 95 передал 37 пакетов данных
Клиент 96 передал 49 пакетов данных
Клиент 97 передал 32 пакетов данных
Клиент 98 передал 65 пакетов данных
Клиент 99 передал 54 пакетов данных
Клиент 100 передал 100 пакетов данных

Общее количество принятых пакетов данных: 4873 из 10000 (48%)

^C

pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15\$

1000 клиентов

В терминале клиента:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ ./с 1000
```

Используем порт по умолчанию: 8888

Используем адрес по умолчанию: 127.0.0.1

Создано 1000 потоков-клиентов

Конец работы программы

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$
```

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$ ./s
```

Используем порт по умолчанию: 8888

Обработан клиент 1

Обработан клиент 4

Обработан клиент 5

<...>

Обработан клиент 992

Обработан клиент 999

Обработан клиент 1000

^C

Клиент 1 передал 95 пакетов данных

Клиент 2 передал 85 пакетов данных

<...>

Клиент 234 передал 100 пакетов данных

Клиент 235 передал 100 пакетов данных

Клиент 236 передал 100 пакетов данных

<...>

Клиент 942 передал 32 пакетов данных

<...>

Клиент 998 передал 55 пакетов данных

Клиент 999 передал 56 пакетов данных

Клиент 1000 передал 54 пакетов данных

Общее количество принятых пакетов данных: 49353 из 100000 (49%)

^C

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/15$
```


Как видно из логов, взаимодействие сервера с 10, 100 и 1000 клиентов проходит достаточно успешно. Отметим некоторые особенности работы UDP-протокола:

- Почти каждый отдельно взятый клиент потерял значительную часть данных при передаче. Около половины всех передаваемых данных было утеряно в целом;
- Клиенты передавали свои данные абсолютно непоследовательно (не в порядке создания).
- Для получения сообщения между сервером и клиентом не требуется установки соединения.
- UDP-протокол работает быстрее, чем TCP.

Можно сделать вывод, что UDP-протокол является ненадежным, но более быстрым способом передачи данных.

16. **Задание:** экспериментально продемонстрируйте разницу между TCP и UDP реализациями

Решение: Разница в работе этих протоколов была продемонстрирована на примере взаимодействия с 10, 100 и 1000 клиентов в заданиях 14 и 15.

Представим сводную таблицу в качестве вывода:

<i>Критерий</i>	ТСР-протокол	UDP-протокол
-----------------	--------------	--------------

<i>Гарантия доставки данных и исправления ошибок</i>	Есть, надежный протокол	Нет, ненадежный протокол
<i>Состояние соединения</i>	Требуется установка соединения для передачи данных	Протокол без соединения
<i>Порядок передачи данных</i>	В порядке создания клиентов	Произвольный
<i>Скорость работы</i>	Ниже	Выше

17. **Задание:** Обеспечьте разделение памяти между независимыми процессами и необходимую синхронизацию для эффективного взаимодействия

Решение: Были написаны программы, реализующие разделенный бинарный семафор, то есть пару семафоров для чтения и для записи, только один из которых имеет значение 1 в любой момент времени. Такая организация позволяет обеспечить синхронизацию, т.е. отсутствие повторного чтения и/или записи одних и тех же данных.

В программе-сервере захватывается семафор для чтения, из него выводятся данные и освобождается семафор для записи. В программе-клиенте захватывается семафор для записи, передаются данные, а затем освобождается семафор для чтения.

Исходный код: Файл server.c

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include "shm.h"

int shmemory;
int semaphore;

void intHandler() {
    // удаляем shm и семафоры
    if (shmctl(shmemory, IPC_RMID, 0) < 0) {
        printf("Ошибка удаления общей памяти\n");
        exit(1);
    }
    if (semctl(semaphore, 0, IPC_RMID) < 0) {
        printf("Ошибка удаления семафора\n");
        exit(1);
    }

    printf("\nЗавершение работы программы\n");
    exit(0);
}

int main() {
    const char keyFile[] = "keyFile";
    printf("Создание сервера\n");
    // на основе файла и proj_id, который может задаваться
буквой,
    // создается ключ для обеспечения разделяемой памяти
    key_t key;
    if ((key = ftok(keyFile, 'Q')) < 0) {
        printf("Ошибка создания ключа из файла keyFile и id
'Q'\n");
        exit(1);
    }

    // создаем shm с разрешениями 0666
    if ((shmemory = shmget(key, sizeof(Message), IPC_CREAT |
0666)) < 0) {

```

```

    printf("Ошибка создания общей памяти\n");
    exit(1);
}

// присоединяем shm в наше адресное пространство, NULL
означает,
// что будет автоматически определен и использован подходящий
адрес
Message *p_msg;
if ((p_msg = (Message *) shmat(shmemory, NULL, 0)) < 0) {
    printf("Ошибка при присоединении памяти\n");
    exit(1);
}

// устанавливаем обработчик сигнала
signal(SIGINT, intHandler);

// создаем группу из 2 семафоров, соответствующую ключу key
// первый показывает, что можно читать
// второй показывает, что можно писать
if ((semaphore = semget(key, 2, IPC_CREAT | 0666)) < 0) {
    printf("Ошибка создания семафоров\n");
    kill(getpid(), SIGINT);
}
printf("Создан разделенный бинарный семафор\n");
// устанавливаем семафор для записи в 1, т.е. можно писать
if (semop(semaphore, setWriteEna, 1) < 0) {
    kill(getpid(), SIGINT);
}

// основной цикл работы
for (;;) {

    // ждем пока семафор для чтения не станет 1, т.е. можно
читать
    if (semop(semaphore, readEna, 1) < 0) {
        kill(getpid(), SIGINT);
    }
    // семафор для чтения стал 0
    printf("\nЗахват семафора \"чтение разрешено\"\n");

```

```

        // читаем сообщение от клиента
        printf("Сообщение клиента: %s", p_msg->buf);

        // устанавливаем семафор для записи в 1, т.е. можно писать
        if (semop(semaphore, setWriteEna, 1) < 0) {
            kill(getpid(), SIGINT);
        }
        printf("Освобождение семафора \"запись разрешена\"\\n");
    }
}

```

Файл client.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include "shm.h"

char buf[100];

int main() {
    const char keyFile[] = "keyFile";

    // на основе файла и proj_id, который может задаваться
    // буквой,
    // создается ключ для обеспечения разделяемой памяти
    key_t key;
    if ((key = ftok(keyFile, 'Q')) < 0) {
        printf("Ошибка создания ключа из файла keyFile и id
'Q'\\n");
        exit(1);
    }

    // создаем shm с разрешениями 0666
    int shmemory;
    if ((shmemory = shmget(key, sizeof(Message), 0666)) < 0) {
        printf("Ошибка создания общей памяти\\n");
        exit(1);
    }
}

```

```

    }

    // присоединяем shm в наше адресное пространство, NULL
    означает,
    // что будет автоматически определен и использован подходящий
    адрес
    Message *p_msg;
    if ((p_msg = (Message *) shmat(shmemory, NULL, 0)) < 0) {
        printf("Ошибка присоединения памяти\n");
        exit(1);
    }

    // создается набор семафоров, соответствующий ключу key
    int semaphore;
    if ((semaphore = semget(key, 2, 0666)) < 0) {
        printf("Ошибка создания семафоров\n");
        exit(1);
    }
    printf("Создан разделенный бинарный семафор\n");

    for (;;) {
        printf("\nВведите сообщение или пустую строку для
    завершения работы\n");

        // чтение сообщения
        bzero(buf, 100);
        fgets(buf, 100, stdin);

        // проверка на пустую строку
        if (strlen(buf) == 1 && buf[0] == '\n') {
            break;
        };
        // ждем пока семафор для записи не станет 1, т.е. можно
        писать
        if (semop(semaphore, writeEna, 1) < 0) {
            printf("Ошибка изменения семафора\n");
            exit(1);
        }
        // семафор для записи стал 0
        printf("Захват семафора \"запись разрешена\"\n");
    }

```

```

        // запись сообщения в разделяемую память
        sprintf(p_msg->buf, "%s", buf);
        printf("Сообщение клиента записано в разделяемую
память\n");

        // устанавливаем семафор для чтения в 1, т.е. можно читать
        if (semop(semaphore, setReadEna, 1) < 0) {
            printf("Ошибка изменения семафора\n");
            exit(1);
        }
        printf("Освобождение семафора \\"чтение разрешено\\"\\n");
    }

    return 0;
}

```

Файл shm.h

```

#ifndef SHM_H
#define SHM_H

#include <sys/sem.h>

typedef struct {
    long type;
    char buf[100];
} Message;

// семафоры для чтения
static struct sembuf setReadEna[1] = {0, 1, 0};
static struct sembuf readEna[1] = {0, -1, 0};

// семафоры для записи
static struct sembuf setWriteEna[1] = {1, 1, 0};
static struct sembuf writeEna[1] = {1, -1, 0};

#endif //SHM_H

```

Вывод программы:

В терминале сервера:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$ gcc
server.c -o s
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$ gcc
client.c -o c
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$ ./s
Создание сервера
Создан разделенный бинарный семафор

Захват семафора "чтение разрешено"
Сообщение клиента: It's client 1!
Освобождение семафора "запись разрешена"

Захват семафора "чтение разрешено"
Сообщение клиента: It's client 2!
Освобождение семафора "запись разрешена"

Захват семафора "чтение разрешено"
Сообщение клиента: Hello!
Освобождение семафора "запись разрешена"

Захват семафора "чтение разрешено"
Сообщение клиента: Bye!
Освобождение семафора "запись разрешена"

Захват семафора "чтение разрешено"
Сообщение клиента: Bye-bye!
Освобождение семафора "запись разрешена"
^C
Завершение работы программы
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$
```

В терминале клиента 1:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$ ./c
Создан разделенный бинарный семафор

Введите сообщение или пустую строку для завершения работы
It's client 1!
Захват семафора "запись разрешена"
Сообщение клиента записано в разделяемую память
```


Освобождение семафора "чтение разрешено"

Введите сообщение или пустую строку для завершения работы
Bye!

Захват семафора "запись разрешена"

Сообщение клиента записано в разделяемую память

Освобождение семафора "чтение разрешено"

Введите сообщение или пустую строку для завершения работы

В терминале клиента 2:

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$ ./c
```

Создан разделенный бинарный семафор

Введите сообщение или пустую строку для завершения работы
It's client 2!

Захват семафора "запись разрешена"

Сообщение клиента записано в разделяемую память

Освобождение семафора "чтение разрешено"

Введите сообщение или пустую строку для завершения работы
Hello!

Захват семафора "запись разрешена"

Сообщение клиента записано в разделяемую память

Освобождение семафора "чтение разрешено"

Введите сообщение или пустую строку для завершения работы
Bye-bye!

Захват семафора "запись разрешена"

Сообщение клиента записано в разделяемую память

Освобождение семафора "чтение разрешено"

Введите сообщение или пустую строку для завершения работы

```
pitatir@pitatir-Lenovo-Legion-Y9000P2021H:~/LR6/17$
```

Как видно из логов, программы обеспечивают разделение памяти между серверами и клиентами и необходимую синхронизацию для

эффективного взаимодействия: никакие данные не считываются и не записываются повторно.

Выводы.

Изучены межпроцессные взаимодействия в Linux: разделение памяти и синхронизация сокет UDP и TCP, произведено их сравнение. Научиться применять перечисленные выше операции и файлы успешно применены на практике.

Использованная литература:

Методическое пособие Душутиной Е.В. «Межпроцессное взаимодействие в операционных системах»